

YogiTrack

Yoga H'om Studio Management System

Project Report — Application 1.0

ACS-5423-999 • Spring 2026

Des Dew

Contents

1. Project Overview	3
2. Technology Stack	3
3. Use Cases Implemented	3
Use Case 1: Add/Modify/Delete Instructor.....	3
Use Case 2: Add/Modify/Delete Customer	4
4. Key Design Decisions.....	5
4.1 Unified People List	5
4.2 Name-Check Gate Pattern	5
4.3 Collision-Safe ID Generation	5
4.4 Dual-Layer Validation.....	5
4.5 UI Design	5
5. UML Models	6
5.1 Use Case Diagram.....	6
5.2 Class Diagram	7
5.3 Sequence Diagram — Add Customer	7
6. Deployment	8
Appendix: Generative AI Usage Disclosure	9
Figure 1 - Website Screenshots.....	5
Figure 2 - Use Case Diagram	6
Figure 3 - Sequence Diagram - Add Customer	7

1. Project Overview

Yoga H'om is a yoga studio which serves the surrounding suburbs of Pittsburgh, Pennsylvania. It offers yoga classes and workshops daily to people of all abilities. Currently, it handles its instructor information, customer registration, class package sales and student attendance through non-computerized methods (manual paper based). The need for an automated method became apparent as the number of students at Yoga H'om has increased. This automation will allow for improved operational efficiency and reduce the chance of recording errors.

YogiTrack is a web-based Studio Management Application created to streamline the main day-to-day functions of Yoga H'om. YogiTrack is focused on the initial stage of this project (Application 1.0) – automating Instructor/Customer Management while maintaining simplicity and ease of use throughout, allowing access from any modern internet browser.

2. Technology Stack

YogiTrack is written using modern JavaScript (full stack), chosen to allow rapid application development; to be easily deployed into the cloud; and to have the backing of an active community.

Layer	Technology	Purpose
Frontend	HTML5, CSS3, Vanilla JavaScript	UI pages, forms, dynamic table rendering
Backend	Node.js, Express.js	REST API, routing, business logic
Database	MongoDB, Mongoose	Document storage, schema validation
Deployment	Heroku	Cloud hosting, CI/CD pipeline
Version Control	Git, GitHub	Source control, deployment integration

The Back End has a RESTful API that allows for full CRUD (Create Read Update Delete) Operations for Customers and Instructors. The Front End uses the Fetch API to communicate with the Back End's API. All Records are stored in a NoSQL Database (Mongo Db) as Documents and Mongoose provides schema level validation and enforces the presence of required fields and defined enumerations.

3. Use Cases Implemented

YogiTrack version Application 1.0 includes two main use cases from the YogiTrack use case diagram; both use cases were completed in full and made available via deployment.

Use Case 1: Add/Modify/Delete Instructor

Actor
Manager

Description

The manager adds, edits, or removes instructor records in YogiTrack. Each instructor record contains identifying and contact information used to schedule and communicate with instructors.

Data Fields

- Instructor ID (system-generated, prefix I, e.g. I00001)
- First Name, Last Name
- Address
- Phone (format: 123-456-7890)
- Email
- Preferred Mode of Communication (phone or email)

Basic Flow — Add Instructor

1. The manager navigates to Add Instructor from the home screen.
2. The manager enters the instructor's first and last name and clicks 'Check Instructor Name'.
3. YogiTrack will search for an existing instructor using the same name as entered by the manager. If a duplicate is found, the manager is prompted to confirm they wish to create a new entry.
4. YogiTrack reveals the remaining fields. The manager enters address, phone, email, and preferred contact method.
5. YogiTrack validates all required fields on submission
6. YogiTrack generates a unique Instructor ID (I00001, I00002, ...) based on the highest existing ID.
7. YogiTrack saves the record and displays: "Welcome to Yoga H'om! Your instructor id is I00xxx."
8. The manager will be redirected to the People List.

Modify Flow

The manager selects Edit on any instructor row in the People List. The existing record will be loaded into the edit form by YogiTrack. The manager updates the desired fields and saves. YogiTrack updates the record and returns to the People List.

Delete Flow

The manager selects Delete on any instructor row. YogiTrack presents a confirmation dialog. Upon confirming deletion, the record will be deleted and People List will refresh.

Use Case 2: Add/Modify/Delete Customer

Actor

Manager

Description

The manager creates, updates or deletes client data. Client data contains the client's contact information as well as a class balance that tracks the number of class credit hours the client has.

Data Fields

- Customer ID (system-generated, prefix C, e.g. C00001)
- First Name, Last Name
- Address
- Phone (format: 123-456-7890)
- Email
- Preferred Mode of Communication (phone or email)
- Class Balance (defaults to 0 on creation)

Basic Flow — Add Customer

9. The manager navigates to Add Customer from the home screen.
10. The manager enters first and last name and clicks 'Check Customer Name'.
11. YogiTrack will search for an existing customer using the same name as entered by the manager. If a duplicate is found, the manager is prompted to confirm they wish to create a new entry.
12. YogiTrack reveals the remaining fields. The manager enters address, phone, email, and preferred contact method.
13. YogiTrack validates all required fields on submission.
14. YogiTrack generates a unique Customer ID (C00001, C00002, ...) based on the highest existing ID.
15. Class Balance is initialized to 0.
16. YogiTrack saves the record and displays: "Welcome to Yoga H'om! Your customer id is C00xxx."
17. The manager will be redirected to the People List.

Modify and Delete Flow

Identical to the instructor flows. Edit loads the existing record, saves changes, and returns to the People List. Delete prompts for confirmation before removing the record.

4. Key Design Decisions

4.1 Unified People List

Rather than maintaining separate customer and instructor list pages, Application 1.0 presents a unified People List that combines both record types into a single view. Each row is visually tagged with a color-coded type badge (pink for Customer, lavender for Instructor). This reduces navigation steps and gives the manager a complete picture of all studio members at a glance.

The People List supports per-column filtering on all fields, allowing the manager to quickly narrow results by type, preferred contact, name, or any other attribute without leaving the page.

4.2 Name-Check Gate Pattern

Add Customer and Add Instructor each use a two-stage process. First, you enter just the name and click the Check Name button. Then, if the name has already been entered, the rest of the information requested will appear. In this way, YogiTrack can confirm whether another person's name has been entered prior to requiring you to spend time entering additional details. If a duplicate is confirmed, a pop-up window will ask you if you want to continue with adding the new record or cancel it altogether, providing you with flexibility since it is possible that two individuals could have the same name.

4.3 Collision-Safe ID Generation

YogiTrack generates unique identifiers for Customers and Instructors by finding the current maximum identifier used in the MongoDB and then increments that value instead of simply counting documents. Since we do not reuse identifiers after they have been assigned to deleted records, this makes sure that no two identifiers will ever match.

4.4 Dual-Layer Validation

Each field that is required in either table is validated twice: once via HTML5 native validation upon form submission on the front-end and again by the Express controller on the back end before anything is written to MongoDB. Finally, the Mongoose schema also requires each field to meet specific requirements and limits the choices available for the "preferredContact" to those specified within our "enum": ("phone", "email"), thus we ensure that our data is valid regardless of how our API is called.

4.5 UI Design

The YogiTrack design utilizes a soft-pastel pink/lavender color scheme which is reminiscent of a yoga studio environment, along with clean typography and abundant white space. As such, all pages utilize a singular style sheet entitled styles.css so as to maintain visual consistency throughout the application. The provided tables come with pink headers, neutral-colored background rows which alternate to provide contrast, and green edit buttons/pink delete buttons to minimize confusion.

The screenshot displays the YogiTrack web application interface. At the top, there are two side-by-side forms: 'Add Customer' and 'Add Instructor'. Each form has fields for 'First Name' and 'Last Name', and a 'Check Name' button. Below these forms is a central section with the 'YogiTrack' logo and the text 'Yoga H'om Studio Management'. Underneath the logo are three buttons: 'Add Customer', 'Add Instructor', and 'View All People'. The 'View All People' button leads to the 'People List' table. The table has a pink header and contains two rows of data. The first row is for a Customer named Sandra Dubowski, and the second row is for an Instructor named Margaret Kowalski. Each row has an 'Edit' button (green) and a 'Delete' button (pink).

ID	Type	First Name	Last Name	Address	Email	Phone	Pref. Contact	Balance	Actions
C00001	Customer	Sandra	Dubowski	55 Mount Lebanon Blvd. Pittsburgh, PA 15228	sandra.dubowski@gmail.com	412-555-1021	email	4	Edit Delete
I00001	Instructor	Margaret	Kowalski	412 Shadyside Ave. Pittsburgh, PA 15232	m.kowalski@yogahom.com	412-555-0181	email	---	Edit Delete

Figure 1 - Website Screenshots

5. UML Models

5.1 Use Case Diagram

The following use cases are identified for YogiTrack. Application 1.0 implements the two shaded use cases. The remaining use cases are planned for Application 2.0.



Figure 2 - Use Case Diagram

Actor	Use Case	Status
Manager	Add/Modify/Delete Instructor	Implemented ✓
Manager	Add/Modify/Delete Customer	Implemented ✓
Manager	Add/Modify/Delete Class	Planned — App 2.0
Manager	Add/Modify/Delete Package	Planned — App 2.0
Manager	Record a Sale	Planned — App 2.0
Manager	Publish Class Schedule	Planned — App 2.0
Manager	View Class Attendance	Planned — App 2.0
Manager	Generate Studio Reports	Planned — App 2.0
Instructor	Record Class Attendance	Planned — App 2.0
Instructor	View Class Schedule	Planned — App 2.0
Instructor	Generate Self-Performance Report	Planned — App 2.0

5.2 Class Diagram

The data model for Application 1.0 consists of two Mongoose schema classes stored in MongoDB.

Customer	Instructor
customerId: String	instructorId: String
firstName: String (required)	firstName: String (required)
lastName: String (required)	lastName: String (required)
address: String (required)	address: String (required)
phone: String (required)	phone: String (required)
email: String (required)	email: String (required)
preferredContact: enum [phone, email]	preferredContact: enum [phone, email]
classBalance: Number (default: 0)	(no class balance)

5.3 Sequence Diagram — Add Customer

The following sequence describes the interaction between system components when adding a new customer.

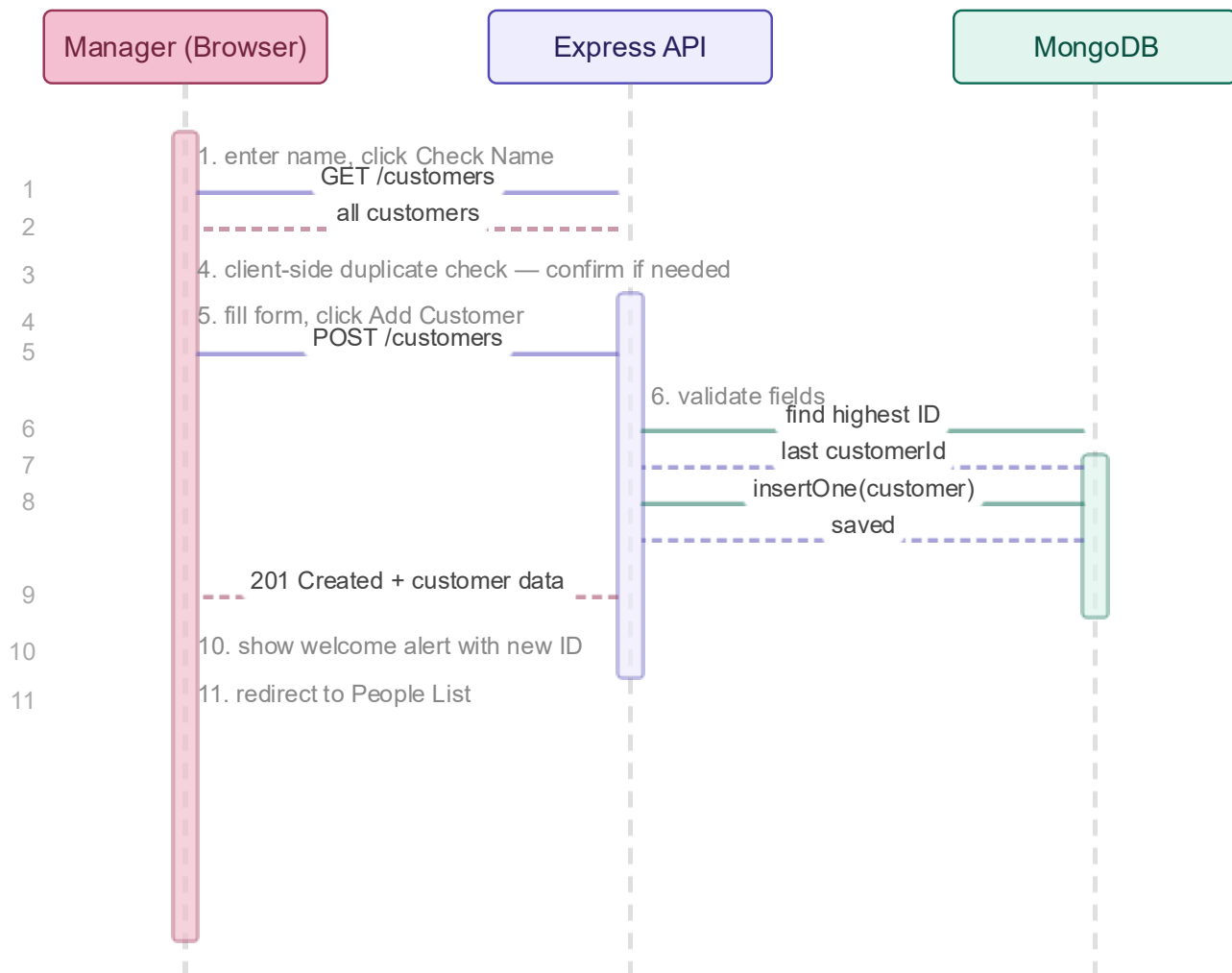


Figure 3 - Sequence Diagram - Add Customer

6. Deployment

YogiTrack Application 1.0 is deployed on Heroku and accessible at:

<https://yogitrack-des-9bc253af7715.herokuapp.com/>

The source code is maintained in a GitHub repository with the Heroku deployment pipeline connected to the main branch. All production changes are deployed by pushing to the main branch.

Item	Detail
Platform	Heroku (Eco Dyno)
Database	MongoDB Atlas (cloud-hosted)
Runtime	Node.js
Deployment	GitHub-connected pipeline, push-to-deploy
Live URL	https://yogitrack-des-9bc253af7715.herokuapp.com/

Appendix: Generative AI Usage Disclosure

In accordance with the course AI policy, I am disclosing my use of generative AI tools throughout this project.

Tools Used

ChatGPT (OpenAI) was used during the initial project setup phase, including configuring the development environment, setting up Git and GitHub, connecting to MongoDB Atlas, and establishing the basic Express server structure.

Claude (Anthropic) was used throughout the development phase after ChatGPT became unresponsive mid-session. Claude assisted with the following:

- Debugging — identifying and fixing issues including non-responsive Edit and Delete buttons caused by DOM rebuilding in a loop, a Customer ID collision bug after record deletions, and a duplicate name header validation issue
- Code review — reviewing controller logic, API response structure, and frontend form behavior
- Planning and design — thinking through the unified People List approach, the two-step name-check form pattern, and the collision-safe ID generation strategy
- UI design — developing the pastel color palette, table layout, and column filter functionality
- Documentation — drafting the project report outline, README, and code comments

All code was tested locally and deployed to Heroku. I reviewed, tested, and can explain the function and logic of every component of the submitted application.